Measurement discipline and relevant information:

Measurement Discipline (System Configuration):

- System Specs:
- CPU: Intel Core i5 1135G7, 4c/8t, 400MHz-4.20GHz
- --> Turbo boost disabled via Intel P_State driver modifications
- --> Pinned to Core 0
- --> Hyperthreading (SMT) is Disabled through BIOS prior to benchmarking
- --> Cache working sets were based on cache size information pulled from intel Ark website, see further below
- RAM: 8GB LPDDR4X 4267MHz (Dual Channel)
- OS: Linux Debian Trixie 13.7.0
- Compiler/Tools:
- GCC 14.2.0 (default with Debian 13)
- Governor: intel cpu_freq
- VSCode IDE

Pre-experiment Hypothesis:

- I expect that throughput will be best when memory access is aligned and using sequential strides (stride of 1). I also suspect that effective bandwidth will sharply drop once we have strides beyond 1 and misaligned memory. Also, sequential access will probably be faster than random access too.

AI Use Note:

- Tools used: Copilot, Gemini
- Copilot was used for:
- --> assisting with benchmark scripting methodology and function calls
- --> discussion around core pinning, testing methodology
- --> discussion around prologue and tail with regards to alignment and tail-handling study
- --> double checking my benchmarking calculations and formulas
- --> second opinion on results
- --> getting compiler flags for scalar vs auto-vectorization vs full targeted SIMD instructions
- --> bouncing ideas off eachother for possible explanations
- --> doing last minute polishing of my report explanations (I wrote the original explanations, but they were rough to read)
- Gemini was used for:
- --> setting up my Windows 11 C/C++ Compiler environment (it was broken before)
- --> helping with random number generation code
- --> Assisting with Linux benchmarking libraries support

Testing Methodology:

- Core Pinning was pinned to Core 0
- Hyperthreading was disabled in BIOS
- Turbo boost is disabled via modifications to the intel cpu_freq governor in Debian, along with linux-powercpu to ensure that clockspeed stays at constant 1.40GHz for core 0. In practice, it stayed exactly at 1.40GHz.
- --> Also all low-power C states were disabled in the script, so it is at C0 100% of the time
- Each kernel/test run was run 20 times to warm up, with results and timing averaged across those 20 runs (runs 3 runs before the 20-run loop in this calculation due to warm ups)
- Benchmarking information collected during benchmarking each kernel run:
- --> Total floating point operations (all runs)
- --> GFLOPS (avg per run)
- --> HW Arithmetic Intensity (FLOPS/Byte)
- --> Elapsed Time (in nanoseconds) across all runs
- --> Average Time (in nanoseconds, per run)

For scalar vs auto-vectorization, I used these compiler flags:

- Scalar: -03 -fno-tree-vectorize
- Auto-vectorization: -03
- (Used for plots #2-5) Targeted SIMD: -03 -mavx512vl (this adds vector length support too)

Cache working set sizes: (these sizes fit within these cache hiarchies)

- For FP32 only:
- --> 4,096 Floats : ~16KB (L1 I, per core)
- --> 9,216 Floats : ~36KB (L1 D, per core)
- --> 262,144 Floats : ~1MB (L2, per core)
- --> 1,572,864 Floats : ~6MB (L3, Shared)
- --> 16,777,216 Floats : ~64MB (DRAM)
- For FP64: removed implementation

According to Intel's website for the I5 1135G7 + some linux commands, I was able to derive the cache hierarchy for this CPU:

- L1-I: 128KiB (32KiB per core)
- L1-D: 192KiB (48KiB per core)
- L2 cache: 5MiB (1.25MiB per core)
- L3 "Smart" cache: 8MiB (shared)

Alignment options chosen for comparison:

- 0: Aligned
- 7: Misaligned

- This is the starting element offset from the original base address. so misaligned is 7 elements off from the original base address.

Strides chosen for sweep:

- 1: default
- 2,4,8
- This is how many elements in the array/vector that it runs over in granularity. so 1 means it strides 1 element at a time (which means it doesn't skip over elements). so 2 means it strides every other element (so element 1 to element 3 to ...)

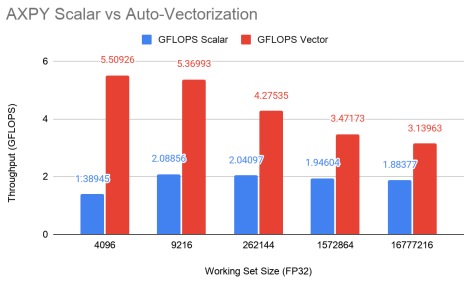
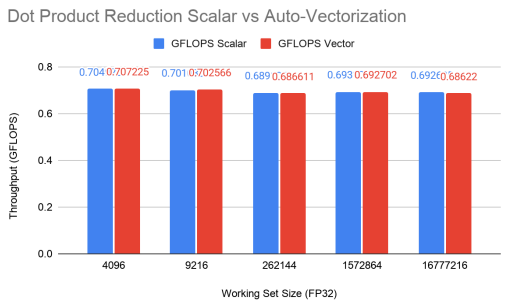
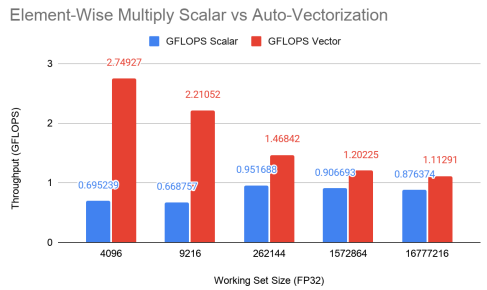
Datatypes chosen:

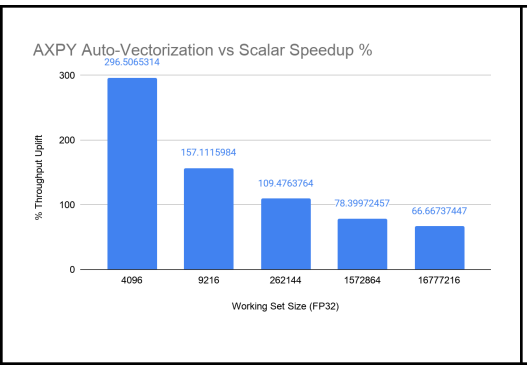
- FP32 (float) : default
- FP64 (double) : chosen as the comparison

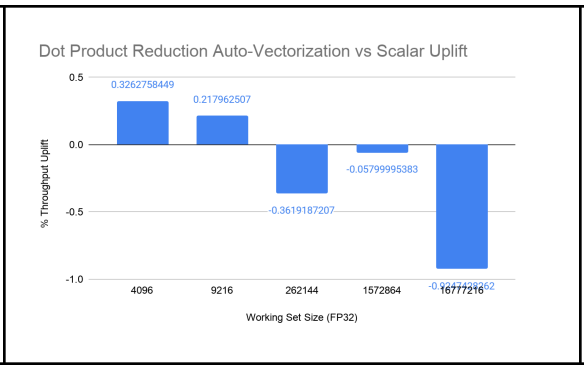
Report + Required Experiments and Plots:

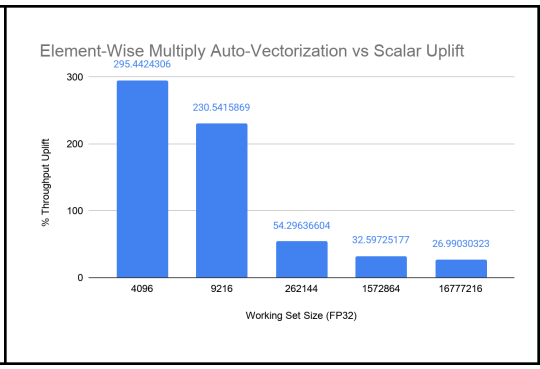
Pre-experiment hypothesis: I expect that throughput will be best when memory access is aligned and using sequential strides (stride of 1). I also suspect that effective bandwidth will sharply drop once we have strides beyond 1 and misaligned memory. Also, sequential access will probably be faster than random access too.

1. Scalar baseline vs auto-vectorization:

AXPY:	Dot Product Reduction:	Element-Wise Multiply:
		



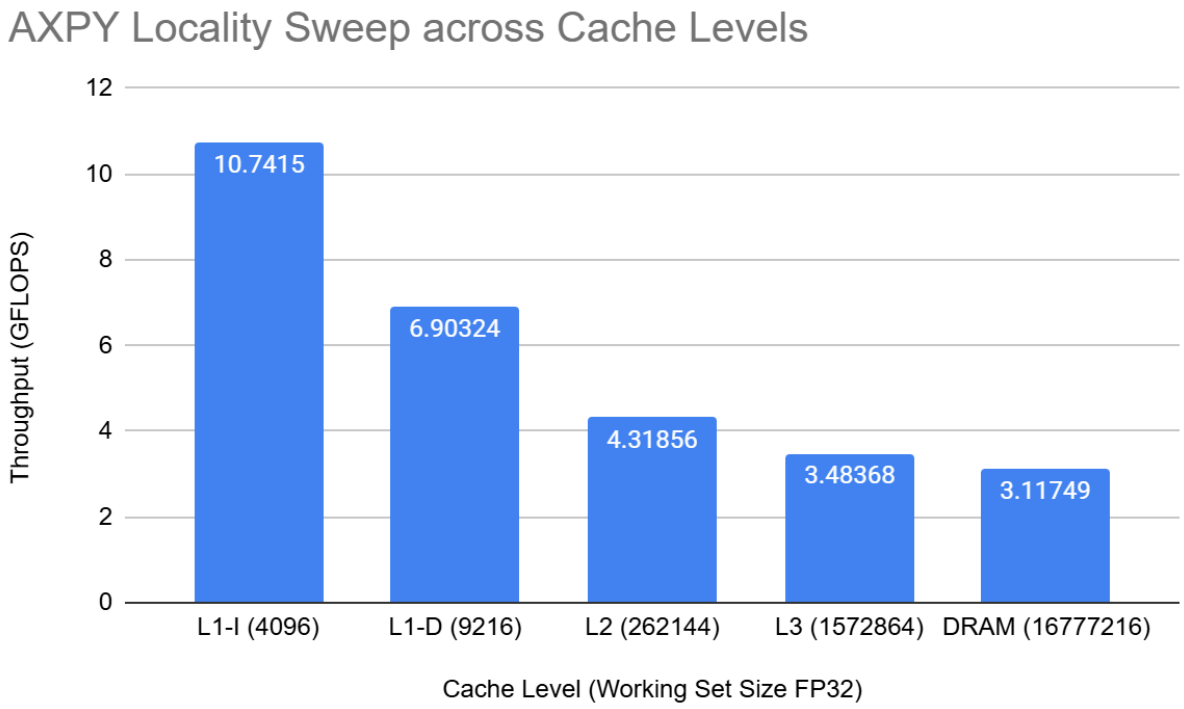




As expected, auto-vectorization significantly improved the performance of the AXPY and Element-Wise Multiply kernels. Both kernels achieved throughput improvements approaching 300% relative to their scalar implementations. These kernels consist primarily of independent operations and therefore map efficiently onto SIMD hardware.

The Dot Product Reduction kernel behaved differently. Initially, I suspected that the loop had failed to auto-vectorize; however, compiler verification confirmed that vectorized code was generated successfully. Despite this, throughput remained within approximately ±1% of the scalar implementation and even became slightly worse once the working set exceeded the L2 cache. This suggests that successful SIMD code generation alone does not guarantee meaningful speedup. The reduction operation requires partial sums to be accumulated into a single final value, introducing dependencies that limit the amount of exploitable parallelism compared to the other kernels.

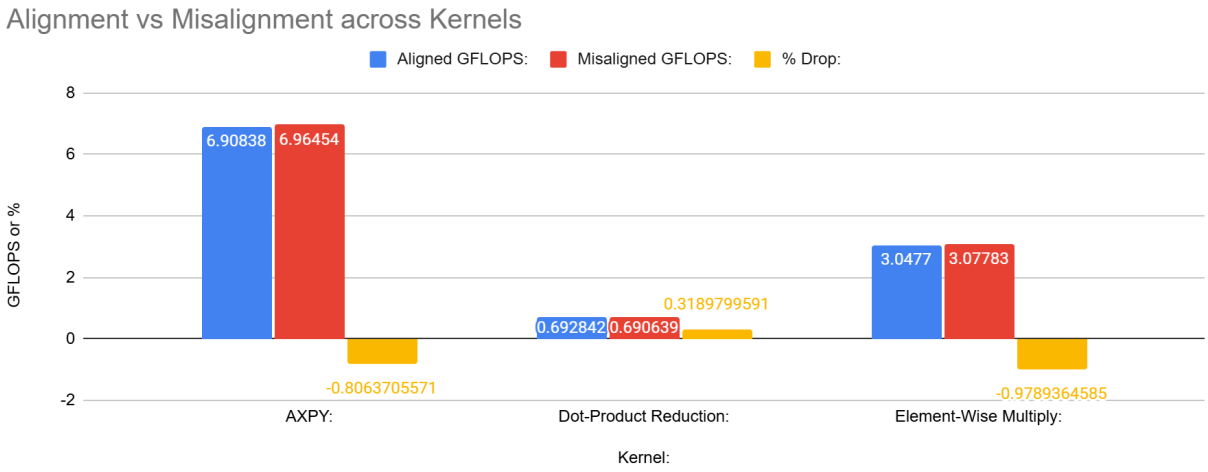
2. Locality sweep:



This kernel showed the most notable downward trend when it comes to performance crossing cache levels. As we go to bigger and slower caches, our throughput in GFLOPs drops dramatically. This is due to our kernel becoming more memory-bound as it uses slower caches with higher latency. SIMD performance drops less aggressively as it crosses from L3 to DRAM.

The arithmetic intensity is constant across cache levels, which suggests that memory constraints are driving the drops in throughput rather than the computational workload.

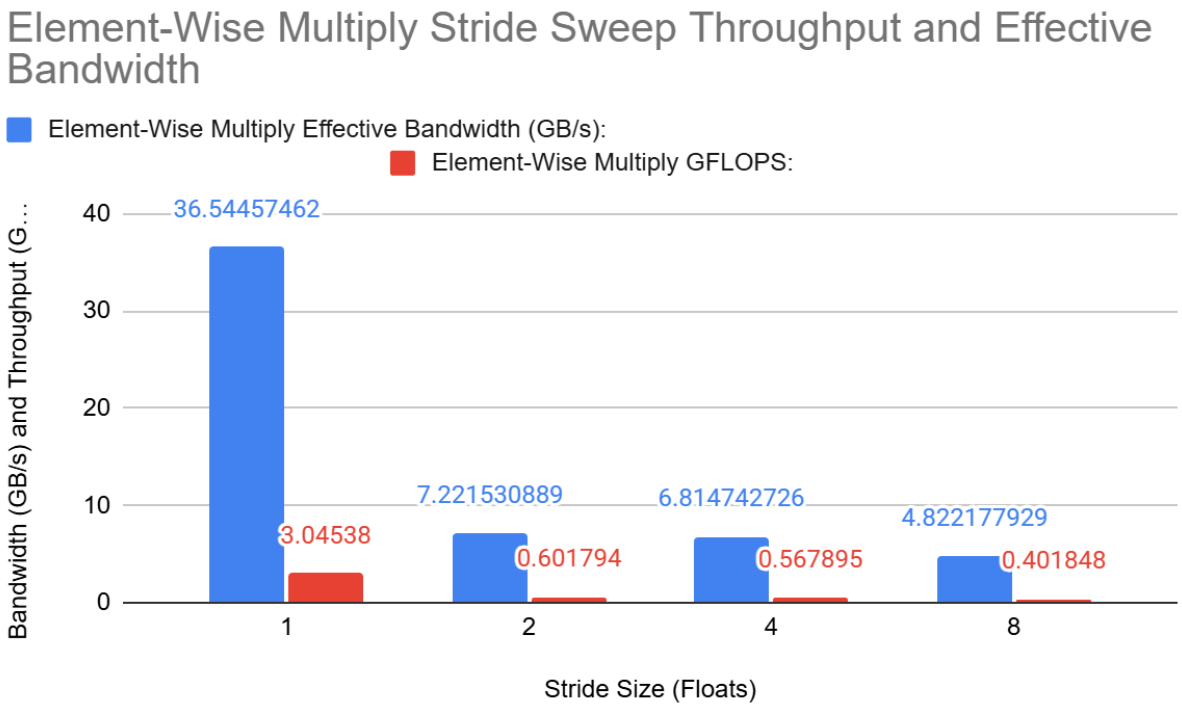
3. Alignment and tail-handling study:



This result was unexpected. I initially anticipated that misalignment would introduce a noticeable performance penalty due to additional prologue/epilogue handling and unaligned memory accesses. However, the measured difference between aligned and misaligned execution remained below 1% for all kernels.

The misalignment offset was seven FP32 elements. Despite this offset, throughput remained nearly identical. These measurements suggest that modern AVX-capable processors handle unaligned loads very efficiently and that any overhead associated with compiler-generated prologue and epilogue code is negligible relative to total execution time. For these workloads, alignment effects were substantially smaller than cache-locality and bandwidth effects.

4. Stride/gather study:



Increasing stride significantly reduced both effective bandwidth and throughput. Modern memory systems transfer data in fixed-size cache lines rather than individual elements. On the test platform, each cache line contains 64 bytes, equivalent to 16 FP32 values.

With a stride of 1, all 16 values within a cache line are used. With a stride of 2, only 8 of the 16 values are useful. As stride increases further, progressively larger portions of each cache line become unused. Consequently, additional memory traffic is required to perform the same amount of computation.

This reduction in spatial locality decreases effective bandwidth and increases memory-access overhead, resulting in a substantial reduction in SIMD throughput. The results demonstrate that

memory-access patterns can have a major impact on performance even when the arithmetic workload remains unchanged.

The reason this reduces efficiency is due to it increasing the number of memory accesses needed to perform the same number of calculations, hence also increasing the latency penalty from increased memory accesses. This decreases the effective bandwidth and increases latency, which drops SIMD throughput efficiency dramatically.

5. Data-type comparison:

(AVX-512, 512bit)	Lane Count:	AXPY Arithmetic Intensity (FLOPS/Byte):	AXPY Operation Cost (FLOPS/element):	Dot-Product Reduction Arithmetic Intensity (FLOPS/Byte):	Dot-Product Reduction Operation Cost (FLOPS/element):	Element-Wise Multiply Arithmetic Intensity (FLOPS/Byte):	Element-Wise Multiply Operation Cost (FLOPS/element):
FP32	16	0.166667	2	0.25	2	0.0833333	1
FP64	8	0.0833333	2	0.125	2	0.0416667	1

The operation count per element remains unchanged between FP32 and FP64. However, arithmetic intensity and SIMD lane count differ substantially.

Because AVX-512 vectors are fixed at 512 bits, FP32 operations can process sixteen values per vector while FP64 operations can process only eight values per vector. FP64 therefore provides half as much data-level parallelism within a single SIMD instruction.

In addition, FP64 doubles the amount of data that must be transferred for each element. As a result, arithmetic intensity is reduced by approximately one half relative to FP32. This pushes workloads further toward the memory-bound region of the roofline model. Conversely, FP32 provides greater SIMD parallelism and requires less memory traffic per element, allowing workloads to operate closer to the compute-bound region.

6. Vectorization verification:

I checked by using the compile flags: -march-native -fopt-info-vec-all
What -fopt-info-vec-all does is that it tells you all information regarding auto-vectorization for all loops and code, and tells you if it failed or succeeded in auto-vectorization.
What -march-native does is that it tells the compiler which SIMD instructions it is using for auto-vectorization when reporting auto-vectorization hits and misses.

Mine looked confusing at first, but I was able to determine that all three kernels were auto-vectorizing correctly.

These were the compilation commands I used, the auto-vectorized one is the third command:

```
#compiling program (deliverables 2-5) with SIMD flags enabled
gcc -O3 -mavx512vl project1.cpp -lstdc++ -lm -o project1.exe 2>&1 | tee compilation.txt #SIMD flags

#compiling program (deliverable 1) 2 times, one with scalar flags and one with auto-vectorization flags enabled
gcc -O3 -fno-tree-vectorize project1Scalar.cpp -lstdc++ -lm -o project1Scalar.exe 2>&1 | tee scalarCompilation.txt #scalar flags
gcc -O3 -march=native -fopt-info-vec-all project1Vector.cpp -lstdc++ -lm -o project1Vector.exe 2>&1 | tee vectorCompilation.txt #a
```

First, here are the auto-vectorized kernels (with line numbers):
AXPY:

```
53  void AXPY_SIMD(float a, std::vector<float>& x, std::vector<float>& y, size_t strides, size_t alignmentOffset){/
54      //pointer version, allows for lower overhead and custom strides
55      size_t xsize = x.size();
56      size_t ysize = y.size();
57      float* __restrict xpointer = x.data() + alignmentOffset;
58      float* __restrict ypointer = y.data() + alignmentOffset;
59      if((xsize == ysize) && (alignmentOffset < ysize)){
60          for(size_t i = 0; i < ysize - alignmentOffset; i += strides){//Main Vector Loop
61              ypointer[i] += a * xpointer[i];
62          }
63      }
64  }
```

Dot Product Reduction:

```
68  float DotProductReduction_SIMD(std::vector<float>& x, std::vector<float>& y, size_t strides, size_t alignmentOffset){
69      //pointer version, allows for lower overhead and custom strides
70      size_t xsize = x.size();
71      size_t ysize = y.size();
72      float* __restrict xpointer = x.data() + alignmentOffset;
73      float* __restrict ypointer = y.data() + alignmentOffset;
74      if((xsize == ysize) && (alignmentOffset < ysize)){
75          float s = 0.0f;
76          for(size_t i = 0; i < ysize - alignmentOffset; i += strides){//Main Vector Loop
77              s += ypointer[i] * xpointer[i];
78          }
79          return s;
80      }
81      return 0.0f;
82  }
```

Element-wise Multiply:

```
86  void ElementWiseMultiply_SIMD(std::vector<float>& x, std::vector<float>& y, std::vector<float>& z, size_t strides, size_t alignmer
87      //pointer version, allows for lower overhead and custom strides
88      size_t xsize = x.size();
89      size_t ysize = y.size();
90      size_t zsize = z.size();
91      float* __restrict xpointer = x.data() + alignmentOffset;
92      float* __restrict ypointer = y.data() + alignmentOffset;
93      float* __restrict zpointer = z.data() + alignmentOffset;
94      if((xsize == ysize) && (ysize == zsize) && (alignmentOffset < zsize)){
95          for(size_t i = 0; i < zsize - alignmentOffset; i += strides){//Main Vector Loop
96              zpointer[i] = xpointer[i] * ypointer[i];
97          }
98      }
99  }
```

And here is the proof in the compilation output that they are auto-vectorized:
AXPY:

```
project1Vector.cpp:60:29: optimized: loop vectorized using 32 byte vectors
project1Vector.cpp:60:29: optimized:  loop versioned for vectorization because of possible aliasing
project1Vector.cpp:60:29: optimized: loop vectorized using 16 byte vectors
project1Vector.cpp:60:29: missed: couldn't vectorize loop
project1Vector.cpp:60:29: missed: not vectorized: number of iterations cannot be computed.
project1Vector.cpp:53:6: note: vectorized 1 loops in function.
```

Dot Product Reduction:

```
project1Vector.cpp:76:29: optimized: loop vectorized using 32 byte vectors
project1Vector.cpp:76:29: optimized: loop vectorized using 16 byte vectors
project1Vector.cpp:76:29: missed: couldn't vectorize loop
project1Vector.cpp:76:29: missed: not vectorized: number of iterations cannot be computed.
project1Vector.cpp:68:7: note: vectorized 1 loops in function.
```

Element-wise Multiply:

```
project1Vector.cpp:95:29: optimized: loop vectorized using 32 byte vectors
project1Vector.cpp:95:29: optimized:  loop versioned for vectorization because of possible aliasing
project1Vector.cpp:95:29: optimized: loop vectorized using 16 byte vectors
project1Vector.cpp:95:29: missed: couldn't vectorize loop
project1Vector.cpp:95:29: missed: not vectorized: number of iterations cannot be computed.
project1Vector.cpp:86:6: note: vectorized 1 loops in function.
```

What's happening here is that the compiler generated 32-byte vector instructions for the loop and made versions of the loop to handle aliasing and remainder iterations. Some of the later vectorization attempts failed due to the runtime-variable stride preventing the compiler from computing loop iterations at compile time. This isn't an issue because the other vectorized loop versions for the same loops were correctly generated and chosen by the compiler.

I originally thought that dot product reduction was supposed to benefit from auto-vectorization, but this is a clear case where it doesn't help. I suspect it is the way I implemented the dot product code that struggles to take advantage of auto-vectorization. I suspect that the reduction portion of the kernel is what causes auto-vectorization benefits to break down, as it has to collapse the vector into a single final value, which serializes the computations.

7. Roofline interpretation:

AXPY:	Working Set:	HW Arithmetic Intensity (FLOPS/Byte):	GFLOPS:	Effective Bandwidth (GB/s):
L1-I	4096	0.166667	10.7415	64.4488711
L1-D	9216	0.166667	6.90324	41.41935716
L2	262144	0.166667	4.31856	25.91130818
L3	1572864	0.166667	3.48368	20.9020382
DRAM	16777216	0.166667	3.11749	18.70490259

The AXPY kernel has an arithmetic intensity of approximately 0.167 FLOPs/Byte, placing it firmly in the low-arithmetic-intensity region of the roofline model. The measured results show that arithmetic intensity remains constant while both effective bandwidth and throughput decrease as the working set moves through the memory hierarchy.

Throughput falls from approximately 10.7 GFLOPs in L1 cache to approximately 3.1 GFLOPs in DRAM, while effective bandwidth decreases from roughly 64 GB/s to 19 GB/s. This trend

indicates that performance increasingly follows the available memory bandwidth rather than the available computational throughput.

These observations are consistent with roofline-model predictions for memory-bound workloads. As data moves farther from the processor and must be fetched from slower portions of the memory hierarchy, memory bandwidth becomes the dominant performance constraint and limits achievable throughput.

8. Include one negative example:

Dot-Product Reduction:	Working Set:	HW Arithmetic Intensity (FLOPS/Byte):	GFLOPS:	Effective Bandwidth (GB/s):
L1-I	4096	0.249998	0.70593	2.82374259
L1-D	9216	0.2499999	0.699699	2.79879712
L2	262144	0.25	0.690066	2.760264
L3	1572864	0.25	0.689951	2.759804
DRAM	16777216	0.25	0.689777	2.759108

The Dot Product Reduction kernel serves as a useful negative example. Despite successful auto-vectorization, throughput remained approximately 0.69 GFLOPs across all cache levels and closely matched the scalar implementation.

Unlike AXPY and Element-Wise Multiply, dot-product reduction requires a final accumulation step that combines many intermediate results into a single output value. This introduces dependencies that reduce the effectiveness of SIMD execution. The relatively constant throughput and low effective bandwidth across cache levels suggest that computational dependencies, rather than memory bandwidth, are the primary performance limitation for this kernel.

This demonstrates that successful vectorization alone is not sufficient to guarantee substantial speedup. Algorithmic structure can ultimately determine whether SIMD parallelism translates into meaningful performance gains.